1.a.

```python
class ImageStandardizer:
    """Standardize a batch of images to mean 0 and variance 1.

    The standardization should be applied separately to each channel.
    The mean and standard deviation parameters are computed in `fit(X)` and
    applied using `transform(X)`.

    X has shape (N, image_height, image_width, color_channel)
    """

    def __init__(self) -> None:
        """Initialize mean and standard deviations to None."""
        self.image_mean = None
        self.image_std = None

    def fit(self, X: npt.NDArray) -> None:
        """Calculate per-channel mean and standard deviation from dataset X."""
        # TODO: 1(a) - Complete this function
        X = np.asarray(X)

        self.image_mean = X.mean(axis=(0, 1, 2))
        self.image_std = X.std(axis=(0, 1, 2))

    def transform(self, X: npt.NDArray) -> npt.NDArray:
        """Return standardized dataset given dataset X."""
        # TODO: 1(a) - Complete this function

        # won't work if fit() isn't called first
        if self.image_mean is None or self.image_std is None:
            raise RuntimeError("Call fit(X) before transform(X).")

        return (X - self.image_mean) / self.image_std
```

1.a.ii.

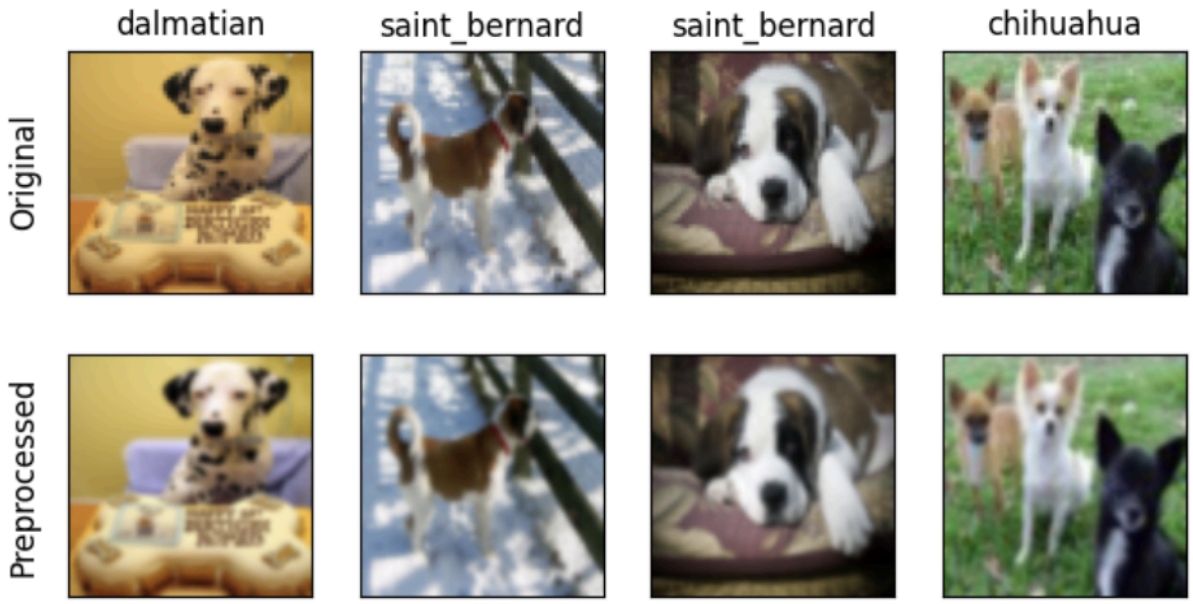
Mean: R: 125.55 G: 118.788 B: 94.766

Std: R: 64.413 G: 61.379 B: 64.247

1.a.ii.

We only take the mean and standard deviation from the training data because that's the only data the model is allowed to see while learning. This will ensure that when we test against the validation and test sets that it is a real life prediction of how the model will generalize to unseen data.

1.b.



We see a set of softer and blurrier images.

2.a.

Layer 0: 0 params

Layer 1:

output channels * (weights per filter) + number of biases

16 * (3*5*5) + 16 = 1216 params

Layer 2: 0 params

Layer 3:

output channels * (weights per filter) + number of biases

64 * (16*5*5) + 64 = 25664 params

Layer 4: 0 params

Layer 5:

output channels * (weights per filter) + number of biases

8 * (64*5*5) + 8 = 12808 params

Layer 6: 2 * 32 + 2 = 66 params

Total learnable params = 39,754

2.b.

```python
class Target(nn.Module):
    def __init__(self) -> None:
        """Define model architecture."""
        super().__init__()

        # TODO: 2(b) - define each layer

        # padding of 2 satisfies same for all three
        self.conv1 = nn.Conv2d(3, 16, 5, kernel_size=2, padding=2)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(16, 64, 5, kernel_size=2, padding=2)
        self.conv3 = nn.Conv2d(64, 8, 5, kernel_size=2, padding=2)
        self.fc_1 = nn.Linear(32, 2)

        # self.conv3 = torch.nn.Conv2d(64, 64, 5, 2, padding=2)
        # self.fc_1 = torch.nn.Linear(256, 2)

        self.init_weights()

    def init_weights(self) -> None:
        """Initialize model weights."""
        torch.manual_seed(42)
        for conv in [self.conv1, self.conv2, self.conv3]:
            # TODO: 2(b) - initialize the parameters for the convolutional layers
            v = sqrt(1.0 / (5 * 5 * conv.in_channels))
            nn.init.normal_(conv.weight, mean=0.0, std=v)
            nn.init.constant_(conv.bias, 0.0)

        # TODO: 2(b) - initialize the parameters for [self.fc_1]
        v = sqrt(1.0 / self.fc_1.in_features)
        nn.init.normal_(self.fc_1.weight, mean=0.0, std=v)
        nn.init.constant_(self.fc_1.bias, 0.0)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        N, C, H, W = x.shape

        # TODO: 2(b) - , forward pass
        x = F.relu(self.conv1(x))
        x = self.pool(x)
        x = F.relu(self.conv2(x))
        x = self.pool(x)
        x = F.relu(self.conv3(x))

        # flatten before putting through fully connected layer
        x = torch.flatten(x, 1)
        x = self.fc_1(x)

        return x
```

2.c.

```python
def predictions(logits: torch.Tensor) -> torch.Tensor:
    """Determine predicted class index given logits.

    args:
        logits: The model's output logits. It is a 2D tensor of shape (batch_size, num_classes).

    Returns:
        the predicted class output that has the highest probability. This should be of size (batch
    """
    # TODO 2(c) - implement predictions
    return torch.argmax(logits, dim=1)
```

2.d.

```python
    # TODO: 2(d) - define loss function, and optimizer
    criterion = torch.nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

2.e.

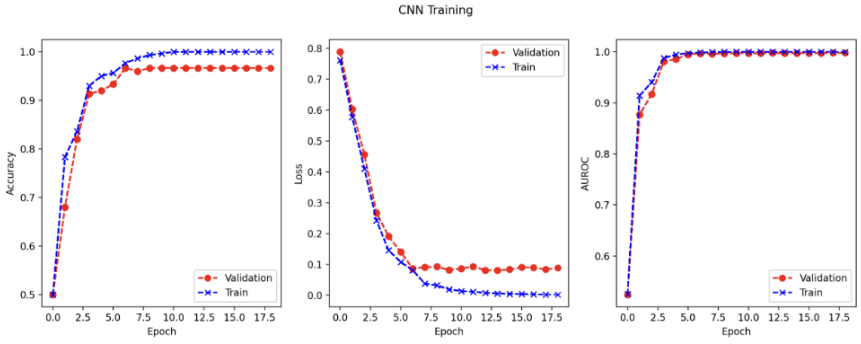
```
264     for i, (X, y) in enumerate(data_loader):
265         # TODO 2(e) - implement training steps:
266         # 1. Reset optimizer gradient calculations
267         # 2. Get model predictions
268         # 3. Calculate loss between model prediction and true labels
269         # 4. Perform backward pass
270         # 5. Update model weights
271 
272         model.train()
273 
274     for X, y in data_loader:
275         optimizer.zero_grad()
276         logits = model(X)
277         loss = criterion(logits, y)
278         loss.backward()
279         optimizer.step()
280 
```

2.f.i.

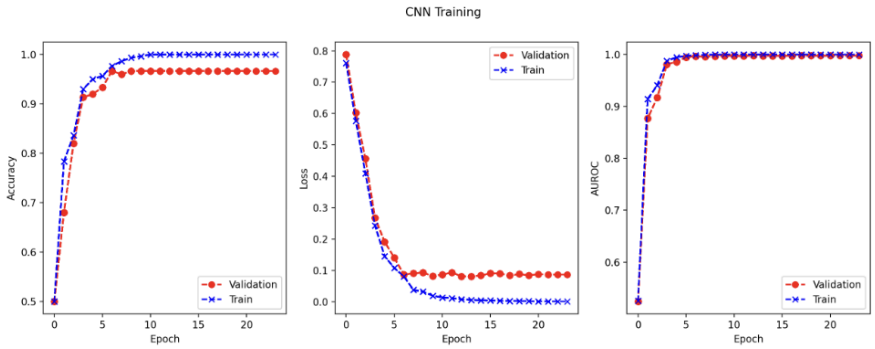
The validation loss wiggles because the model sees different mini-batches of data each epoch, and the randomness in which samples are grouped together can temporarily push the loss up even if training is going well overall. The optimizer also makes updates based only on the current batch rather than the entire dataset, which introduces noise into each step. Since the model starts from randomly initialized weights, early training is especially unstable and sensitive to batch composition. All of this randomness naturally leads to small fluctuations in the validation loss.

2.f.ii.

With patience=5, the model stopped at epoch 18. With patience=10, the model stopped at epoch 23. The two different stopping points appear to do about the same. Looking at the actual epochs though, the model with patience=10 goes 5 more epochs, which means all of the additional epochs past patience=5 are worse than our best validation accuracy. Therefore, I would say the model with patience=5 is better.



patience = 5



patience = 10

2.f.iii.

2 * 2 * 64 = 256

	Epoch	Training AUROC	Validation AUROC
8 filters:	13	1.0	0.9975
64 filters:	8	1.0	0.9959

The model with a 64 layer output from the third convolution layer is more complex and converges faster. Looking at the validation AUROC, it seems as though the 8 layer model does slightly better generalizing to unseen data. This adds up because the variance of the more complex model is expected to rise slightly as we add complexity, which is what we did here. This leads to slightly worse generalization which is shown in the results.

2.g.i.

	Training	Validation	Testing
Accuracy	1.0	0.9667	0.56
AUROC	1.0	0.9975	0.7668

2.g.ii.

We see some evidence of overfitting with respect to training vs validation accuracy and AUROC, but it doesn't seem to be doing too poorly. The models inability to effectively generalize unseen data in the testing accuracy and AUROC show the model's tendency to overfit.

2.g.iii.

The positive labeled data has a red and black box in the upper right hand corner. It is possible that the model learned that the black box in the upper right hand corner was the key indication of whether or not the dog is a collie and that the black and red boxes are not consistent against the two breeds in the test set.

3.a.

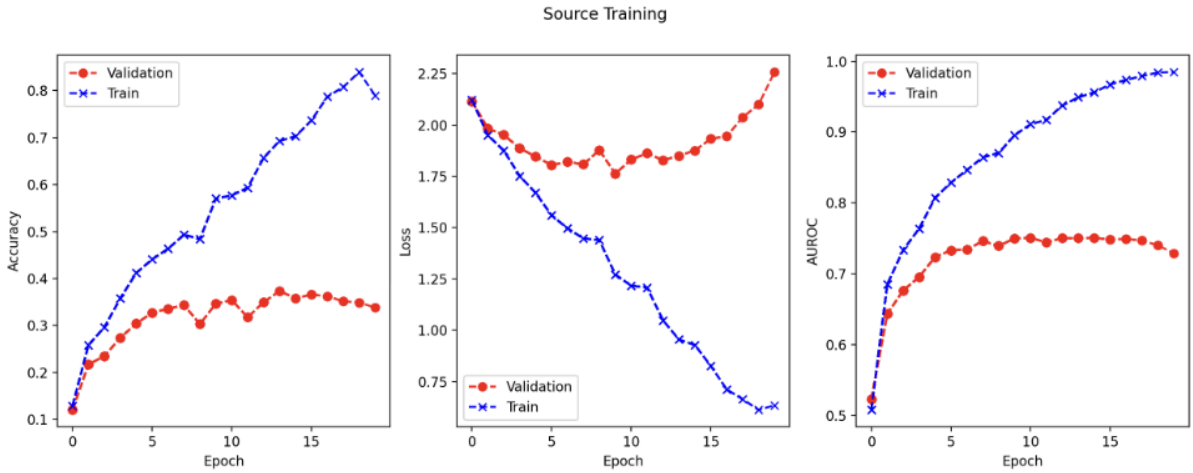
```python
class Source(nn.Module):
    def __init__(self) -> None:
        """Define model architecture."""
        super().__init__()

        # TODO: 3(a) - define each layer
        self.conv1 = nn.Conv2d(3, 16, 5, kernel_size=2, padding=2)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(16, 64, 5, kernel_size=2, padding=2)
        self.conv3 = nn.Conv2d(64, 8, 5, kernel_size=2, padding=2)

        # output 8 vs the target's 2
        self.fc1 = torch.nn.Linear(32, 8)

        self.init_weights()

    # same exact implemetation as target
    def init_weights(self) -> None:
        """Initialize model weights."""
        set_random_seed()

        for conv in [self.conv1, self.conv2, self.conv3]:
            # TODO: 3(a) - initialize the parameters for the convolutional layers
            v = sqrt(1.0 / (5 * 5 * conv.in_channels))
            nn.init.normal_(conv.weight, mean=0.0, std=v)
            nn.init.constant_(conv.bias, 0.0)

        # TODO: 3(a) - initialize the parameters for [self.fc1]
        v = sqrt(1.0 / self.fc1.in_features)
        nn.init.normal_(self.fc1.weight, mean=0.0, std=v)
        nn.init.constant_(self.fc1.bias, 0.0)

    # again same implementation as target
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Perform forward propagation for a batch of input examples. Pass the input array
        through layers of the model and return the output after the final layer.

        Args:
            x: array of shape (N, C, H, W)
                N = number of samples
                C = number of channels
                H = height
                W = width

        Returns:
            z: array of shape (1, # output classes)
        """
        N, C, H, W = x.shape

        # TODO: 3(a) - forward pass
        x = F.relu(self.conv1(x))
        x = self.pool(x)
        x = F.relu(self.conv2(x))
        x = self.pool(x)
        x = F.relu(self.conv3(x))

        # flatten before putting through fully connected layer
        x = torch.flatten(x, 1)
        x = self.fc1(x)

        return x
```

3.b.

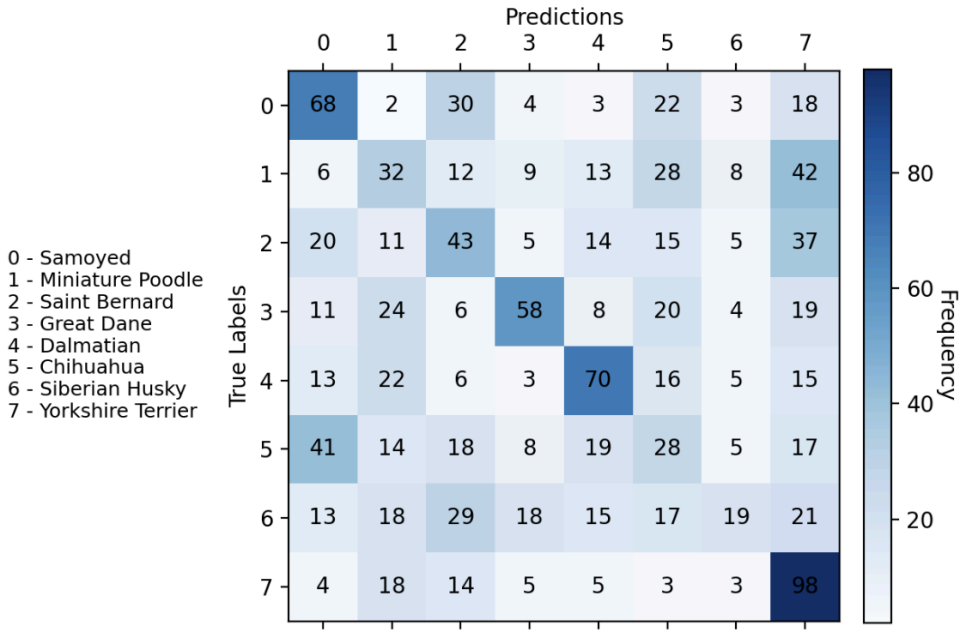
```
33      # TODO: 3(b) - define loss function, and optimizer
34      criterion = torch.nn.CrossEntropyLoss()
35      optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=0.01)
36
```

3.c.

Epoch with lowest val loss = 1.7623 is epoch 9.



3.d.



The breeds the model is most accurate are Samoyed, Great Dane, Dalmation, and Yorkshire Terrier. The least accurate are Miniature Poodle, Chihuahua, and Siberian Husky (worst). Looking into bad performance, it looks like our model misclassifies Miniature Poodles as Yorkies, Chihuahua as samoyeds, and is confused about Siberian Huskies but classifies them the most often as Saint Bernards.

Miniature poodles and yorkies are both small and can have similar coloring. Siberian Huskies and Saint Bernards are both large dogs but do not have many other similar traits. Chihuahuas and Samoyeds have very little resemblance, so our model doesn't seem to have learned the Chihuahua characteristics.

3.e.

```python
def freeze_layers(model: torch.nn.Module, num_layers: int = 0) -> None:
    """
    This function modifies 'model' settings to stop tracking gradients on selected layers.
    The number of convolutional layers to stop tracking gradients for is defined by
    num_layers. You will need to look at PyTorch documentation to implement this function.

    Args:
        model: subclass of nn.Module
        num_layers: int, the number of conv layers to freeze
    """
    # TODO: 3(e) - modify model with the given layers frozen, e.g. if num_layers=2, freeze CONV1
    #       Hint: https://pytorch.org/docs/master/notes/autograd.html

    # nothing to do
    if num_layers == 0:
        return

    conv_count = 0

    for module in model.modules():

        # only freeze conv layers
        if isinstance(module, torch.nn.Conv2d):
            conv_count += 1

            if conv_count <= num_layers:
                for param in module.parameters():
                    param.requires_grad = False

            else:
                break
```

```python
# TODO: 3(e) - define loss function, and optimizer
criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

print("Loading target model with", num_layers, "layers frozen")
model, start_epoch, stats = restore_checkpoint(model, model_name)

axes = make_training_plot("Target Training")

evaluate_epoch(
    axes,
    tr_loader,
    va_loader,
    te_loader,
    model,
    criterion,
    start_epoch,
    stats,
    include_test=True,
)

# initial val loss for early stopping
prev_val_loss = stats[0][1]

# TODO: 3(e) - patience for early stopping, see appendix B
patience = 5
curr_patience = 0
```

3.f.

	Train AUROC	Val AUROC	Test AUROC
Freeze all:	0.8688	0.8779	0.7764
Freeze first two:	1.0	0.9868	0.776
Freeze first:	1.0	0.9963	0.7548
Freeze none:	1.0	0.9989	0.7716
No pretraining:	1.0	0.9975	0.7668

From what we see above, freezing no layers, freezing two layers, and freezing all convolutional layers improved our models' test AUROC. This makes sense because if we only freeze the first layer, we hurt the model because it feeds in only low level edge detectors and does not provide much of the learned representation. Freezing the first two and all three convolutional layers gives the model more transfer representations and we see the best performance with these two (freezing all layers is the best). Freezing no layers also shows improvement because the model has full flexibility to learn and starts at the weights from the learned representations.

The observation that more than freezing two or all layers outperforms freezing no layers checks out because the model retrains some valuable weight information from the transfer because it is given so much flexibility.

4.a.

If the [CLS] embedding were removed, we could still form a classification by applying an average pooling over all the patch embeddings output from the transformer encoder and then feed that pooled vector into the MLP head.

But, we see that this approach was tested by Dosovitskiy and was found to perform worse than using the [CLS] token because the token acts as a learnable aggregator that collects information from all patches through self-attention and learns to represent the entire image in a way optimized for classification. Without it the model relies on averaging and doesn't adaptively weight the important patches.

4.b.

The ViT would lose spatial awareness because the transformer would treat all patches as an unordered set. The model could detect what features exist in the image but not where they are actually located.

4.c.

```
        for sequence in x:
            # Each element in seq_result should be a single attention head's attention values
            seq_result = []
            for head in range(self.num_heads):
                W_k = self.K_mappers[head]
                W_q = self.Q_mappers[head]
                W_v = self.V_mappers[head]

                # Get the given head's k,q,and v representations
                k = W_k(sequence) # (N, d) @ (d, d/H) -> (N, d/H)
                q = W_q(sequence) # (N, d) @ (d, d/H) -> (N, d/H)
                v = W_v(sequence) # (N, d) @ (d, d/H) -> (N, d/H)

                # TODO: 4(c) - Perform scaled dot product self attention, refer to the formula in the spec
                inside = (q @ k.T) / self.scale_factor
                attention = self.softmax(inside) @ v

                # Log the current attention head's attention values
                seq_result.append(attention)

            # For the current sequence (patched image) being processed, combine each attention
            # head's attention values columnwise
            projected_sequence = self.c_proj(torch.hstack(seq_result))
            result.append(projected_sequence)

        return torch.cat([torch.unsqueeze(r, dim=0) for r in result])
```

4.d.

```
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """Forward pass of the Transformer Encoder block with residual connections.

        Args:
            x: Input tensor of shape (batch_size, num_tokens, hidden_d)

        Returns:
            torch.Tensor: Output tensor of the same shape after applying multi-head attention,
            normalization, and MLP.
        """
        # TODO: 4(d) - Define the forward pass of the Transformer Encoder block
        # NOTE: Don't forget about the residual connections!

        # LayerNorm -> MHA
        attn_out = self.multi_head_attention(self.norm1(x))
        x = x + attn_out # residual connection

        # MLP
        mlp_out = self.mlp(self.norm2(x))
        x = x + mlp_out

        return x
```

4.e.

```python
def forward(self, X: torch.Tensor) -> torch.Tensor:
    """
    Forward pass for the Vision Transformer (ViT). N is the number of images in a batch

    Args:
        X: Input batch of images, tensor of shape (N, channels, height, width).

    Returns:
        Tensor: Classification output of shape (batch_size, num_classes).
    """
    B, C, H, W = X.shape

    # Patch images
    patches = patchify(X,self.num_patches)

    # TODO: 4(e) - Get linear projection of each patch to a token (hint: the necessary layers might already be defined!)
    embedded_patches = self.patch_to_token(patches)

    # Add the classification (sometimes called 'cls') token to the tokenized_patches
    all_tokens = torch.stack([torch.vstack((self.cls_token, embedded_patches[i])) for i in range(len(embedded_patches))])

    # Add the positional embedding to the token sequence
    pos_embed = self.pos_embed.repeat(B, 1, 1)
    all_tokens = all_tokens + pos_embed

    # TODO: 4(e) - run the positionaly embedded tokens through all transformer blocks
    # stored in self.transformer_blocks
    for block in self.transformer_blocks:
        all_tokens = block(all_tokens)

    # Extract the classification token and put through mlp
    class_token = all_tokens[:, 0]
    output_logits = self.mlp(class_token)

    return output_logits
```

4.f.

i. 1 x D, so 16 learnable parameters

ii. D_patch = 3 x (64/16) x (64/16) = 3 x 4 x 4 = 48

weight matrix + bias = D_patch x D + D = 48*16 + 16 = 784 learnable parameters

iii.

1. 2 * (weight + bias) = 2*2D = 2*2*16 = 64 learnable parameters

2. total = heads + output projection = 1088 learnable parameters
   heads = 3h(D*(D/h) + (D/h)) = 3(D*D + D) = 816
   output projection = D*D + D = 272

3. total = first + second = 1088 + 1040 = 2128 learnable parameters
   first = weights + biases = in * out + bias = 4*16*16 + 4*16 = 1088
   second = weights + biases = in * out + bias = 4*16*16 + 16 = 1040

total = 2 * (64 + 1088 + 2128) = 6560 learnable parameters in single transformer block

iv. D * number of classes + number of classes = 2*16 + 2 = 34 learnable parameters

v. total = 16 + 784 + 6560 + 34 = 7394 learnable parameters

4.g.

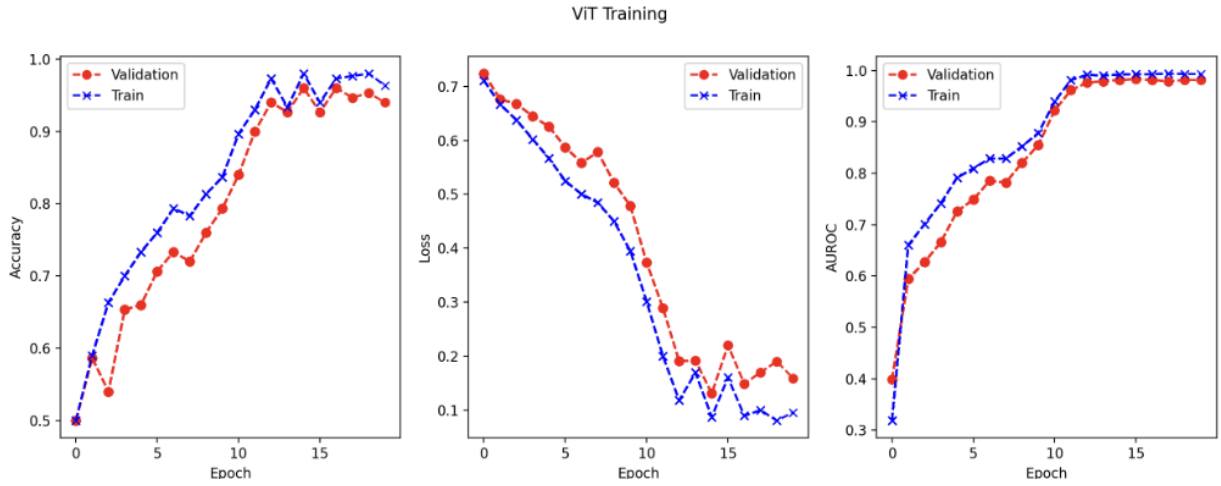
```python
def main():
    """Train ViT and show training plots."""
    set_random_seed()

    # Data loaders
    tr_loader, va_loader, te_loader, _ = get_train_val_test_loaders(
        task="target",
        batch_size=config("vit.batch_size"),
    )

    # TODO: 4(g) - Define the ViT Model according to the appendix D
    model = ViT(
        num_patches=16,
        num_blocks=2,
        num_hidden=16,
        num_heads=2,
        num_classes=2,
        chw_shape=(3, 64, 64)
    )

    # TODO: 4(g) - define loss function, and optimizer
    criterion = torch.nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

    print(f"Number of float-valued parameters: {count_parameters(model)}")

    # Attempts to restore the latest checkpoint if exists
    print("Loading ViT...")
    model, start_epoch, stats = restore_checkpoint(model, config("vit.checkpoint"))

    start_epoch = 0
    stats = []
```

4.h.

epoch 14 best validation loss

Train AUROC: 0.992

Validation AUROC: 0.9819



4.i.

Test AUROC: 0.6452

4.j.

The ViT does not perform as well as our CNN from section 2g in terms of test and validation AUROC. In fact the CNN does noticeably better with a test AUROC of 0.7668 compared to the 0.6452 test AUROC of our ViT. However, the ViT has far less paramters (7,394) compared to the CNN (39,754). This lead to much faster training. In the case where we had more data and limited computing power, the ViT would be a more efficient option.

Challenge:

Challenge Overview:

After testing multiple configurations, I found that a custom-built CNN trained on all available data (train + val + test) with aggressive augmentation and regularization achieved the best performance. The final model reached a validation AUROC of 0.9986 at epoch 31, which was used for my challenge submission.

Challenge Data Augmentation:

Given the small dataset size I knew augmentation was going to be an essential step to prevent overfitting and to improve generalization. I took a probabilistic approach to adding noise to data so that my model was most attuned to normal inputs. I did horizontal flips 25% of the time, and brightness, saturation and contrast jitters at 90%, 70% and 60% respectively. The intuition between probabilities of jittering the different scales was to capture what (in my mind) would be the most likely differences in images of pets. For example brightness likely changes a lot from image to image due to pet photos commonly being inside or outside. The augmentation increased the data size roughly three-fold. During training, images were augmented dynamically so the model rarely saw the same image twice.

Challenge Model Architecture:

For the architecture, I thought back to the discussion of AlexNet vs VGG from this class and my Datasci 315 class. I knew I wanted to have a deep CNN, but with limited image sizes I knew I had to think through it carefully. In order to do this, I used small filters (3x3) with stride=1 and ReLU activations throughout the model. I also used BatchNormalization and pooling in each "chunk" to add lots of parameters without running out of pixels from downsampling too much. For my MLP layer, I found that a two layer architecture performed better than a one layer. I read however that anything larger than two layers for the MLP layer is not very common or helpful. I also used nn.Sequential so that my forward block looked a little cleaner.

Challenge Regularization:

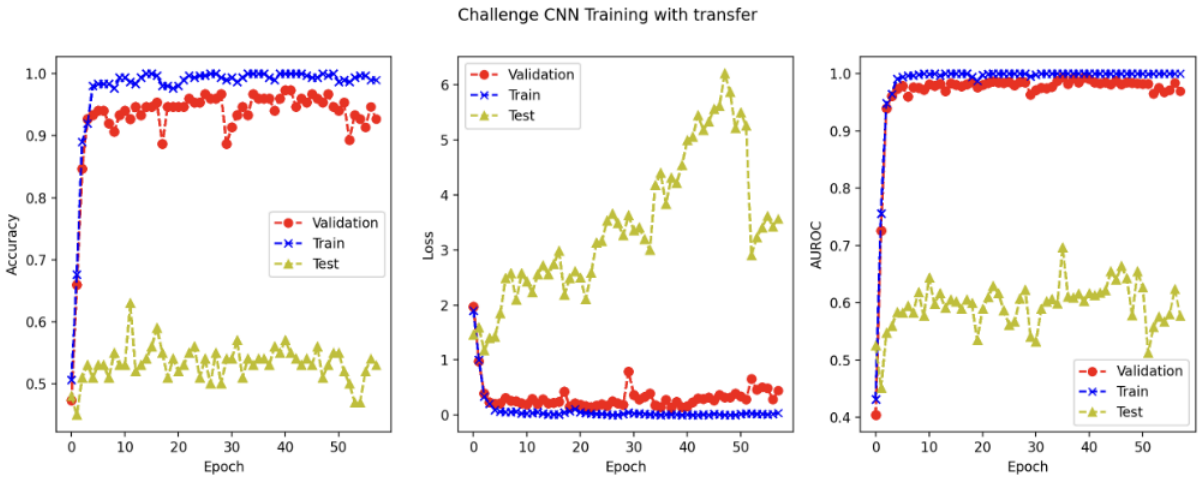
To combat overfitting, I integrated early stopping, dropout (0.5) between dense layers, and weight decay (1e-4) in the Adam optimizer. Additionally, my data augmentation step acted as a strong regularizer. Early in development, I also ran long-duration experiments (400 epochs) to explore the double-descent phenomenon—loss briefly improved and worsened before stabilizing—but these did not outperform shorter, well-regularized training runs. After testing for double descent I was left with an unhappy laptop and possibly some disappointment - sounds like machine learning LOL!

Challenge Hyperparameters:
The main hyperparameter tuning I played around with was dropout, learning rate (in transfer learning), and epochs. For dropout, having a low value made training much faster, but alos lead to worse test performance as expected. A dropout of 0.5 was the happy medium between having a deep CNN and limiting overfitting. As far as epochs, I tuned based on lowest val loss.

Challenge Transfer Learning Experiments:
I spent a lot of time exploring how transfer learning could help out my model, however, I became discouraged after creating 3 models and seeing train test and val AUROC scores of 0.5. Either something was horribly wrong with my code, or there wasn't much semantics to capture. If I had more time I would've explored the possibility of using k-means clustering as a pretraining method, but I can't imagine that would have been too helpful. In the end, I decided to comment out my transfer learning code tuned towards my challenge model. Maybe I bit off too much with augmentation + transfer learning, but in the end I decided to not proceed with transfer learning in my final submission.

Challenge Model Evaluation:
I evaluated each model using accuracy, loss, and AUROC on the training, validation, and test sets. I paid the most attention to the test AUROC as that is the clearest indication of how a model is generalizing to unseen data. My best validation AUROC: 0.9986 (Epoch 31) in my final model, so that's where I loaded my model from for my CSV final submission.

Overall this was a super fun and frustrating challenge, but it's always nice to come up with a model I'm somewhat proud of!



I forgot to change the name of the plot above by the way. There is no transfer learning in my final submission model.